E-Commerce Backend Implementation Pack

Single-store now • Multi-store ready • Implementation blueprint for web, mobile, admin, and operations

This handoff document defines the backend module map, service responsibilities, queues, events, governance controls, and implementation phases for the full 170-screen commerce platform.

Current platform scope	Backend priorities	Delivery outcome
72-screen admin • 50-screen customer web • 48-screen mobile	Transactional integrity • auditability • security monitoring • operational control	A modular monolith implementation pack AI agents and engineers can build against

E-Commerce Backend Implementation Pack

Executive Overview

This document defines the backend implementation pack for a **single-store e-commerce system** that is **future-ready for multi-store expansion**. It consolidates the database architecture, API expectations, admin controls, customer-facing requirements, and mobile/web frontend contracts into a backend delivery blueprint that an AI coding agent or engineering team can implement with much less ambiguity.

The backend is not only an API. It is the operational core of the platform. It must support:

- product catalog, variants, and merchandising
- inventory, warehouses, and stock movement integrity
- carts, checkout, orders, payments, shipping, and refunds
- customer accounts, reviews, support, returns, and preferences
- admin governance, role-based control, auditability, and security monitoring
- queues, webhooks, alerts, incidents, and observability
- clean scaling from single-store now to multi-store later

1. Backend Purpose

The backend must provide:

- a clean domain-driven modular structure
- reliable transactional behavior for orders, payments, and stock
- backend-driven eligibility rules for customer actions
- role-aware admin APIs and sensitive action controls
- append-only operational history for audits, timelines, and investigations
- observability hooks for failures, abuse, and operational incidents
- extension points for multi-store scoping without overengineering the current release

2. Recommended Stack

Recommended implementation stack:

- **Node.js + TypeScript**
- **Express.js**
- **Prisma ORM**
- **PostgreSQL**
- **Redis** for caching, throttling, and background jobs

- **BullMQ** or equivalent for queues and workers
- **S3-compatible storage** for product media and support attachments
- **Paystack or Flutterwave** for Ghana-friendly rollout
- **Resend / Postmark / SendGrid** for transactional email
- **Sentry + structured logging + metrics** for observability

3. Backend Architectural Style

The recommended architecture is a **modular monolith**.

Why:

- faster to build and maintain than microservices at the current stage
- easier transaction management across catalog, inventory, and orders
- simpler deployment model
- lower operational complexity
- still compatible with future extraction of modules into separate services if necessary

3.1 Core design rules

- keep modules isolated by business domain
- use services for business rules, not controllers
- use repositories/data access patterns where helpful, but do not overabstract Prisma
- centralize validation and authorization
- centralize error mapping and response formatting
- make critical business actions idempotent where needed
- write timeline/audit events alongside key state changes

4. High-Level Module Map

Recommended modules:

1. auth
2. users
3. roles-permissions
4. catalog
5. categories
6. brands
7. inventory
8. carts

9. checkout
10. orders
11. payments
12. shipping
13. promotions
14. reviews
15. support
16. notifications
17. content
18. customers
19. reports
20. audit
21. security
22. alerts-incidents
23. integrations
24. system-settings
25. jobs-workers
26. health-observability

5. Recommended Folder Structure

```
src/
  app/
    app.ts
    server.ts
    routes.ts
  config/
    env.ts
    logger.ts
    prisma.ts
    redis.ts
    sentry.ts
  common/
    constants/
    errors/
    middleware/
    utils/
    validators/
    types/
    policies/
  modules/
    auth/
      auth.controller.ts
      auth.service.ts
      auth.routes.ts
      auth.schemas.ts
      auth.types.ts
    users/
    roles-permissions/
    catalog/
    categories/
```

```
    brands/
    inventory/
    carts/
    checkout/
    orders/
    payments/
    shipping/
    promotions/
    reviews/
    support/
    notifications/
    content/
    customers/
    reports/
    audit/
    security/
    alerts-incidents/
    integrations/
    system-settings/
    jobs-workers/
    health-observability/
  workers/
    payment.worker.ts
    notification.worker.ts
    alert.worker.ts
    webhook.worker.ts
  prisma/
    schema.prisma
    migrations/
    seed.ts
```

6. Request Lifecycle Pattern

Each request should typically pass through:

27. route registration
28. auth middleware if needed
29. permission middleware if needed
30. input validation
31. controller
32. service business logic
33. Prisma/database operations
34. event logging / audit / notifications where relevant
35. structured response formatting

6.1 Controller responsibilities

Controllers should:

- parse request context
- call validated service inputs

- avoid business logic
- translate service result into response DTO

6.2 Service responsibilities

Services should:

- enforce business rules
- coordinate transactions
- coordinate writes to related tables
- call external provider adapters
- create audit/timeline/alert events where needed

7. Auth and Identity Module

Responsibilities

- registration
- login
- logout
- password reset
- email verification
- session creation and invalidation
- token issuance and refresh if using JWT-based sessions
- admin session tracking
- login event recording
- lock/suspend enforcement

Key implementation rules

- password hashing with bcrypt or argon2
- secure session/token storage
- separate customer and admin session governance concerns
- login attempts should create `login_events`
- suspicious behavior may create `security_events`

Important endpoints

- `POST /api/auth/register`
- `POST /api/auth/login`
- `POST /api/auth/logout`
- `POST /api/auth/forgot-password`

- `POST /api/auth/reset-password`
- `POST /api/auth/verify-email`

8. Roles, Permissions, and Admin Control

Responsibilities

- role and permission loading
- route-level authorization
- action-level permission checks
- admin user role assignment
- privileged action gating

Recommended policy style

Use permission codes such as:

- `catalog.products.read`
- `catalog.products.write`
- `catalog.products.publish`
- `inventory.adjust`
- `orders.override_status`
- `refunds.approve`
- `support.assign`
- `security.audit.read`
- `incidents.manage`
- `settings.write`

Sensitive action pattern

For critical actions:

- verify permission
- optionally require a reason field
- perform action
- write `admin_action_logs`
- write `audit_logs`
- create downstream alerts where appropriate

9. Catalog Module

Responsibilities

- product CRUD
- variant management
- media metadata
- publishing workflow
- price updates
- SEO fields
- product analytics summary later

Important rules

- product status and visibility are separate concepts
- variants are the main purchasable stock units
- archive instead of delete for important product records
- price changes should be admin-audited

Important endpoints

- `GET /api/products`
- `GET /api/products/:slug`
- `GET /api/admin/products`
- `POST /api/admin/products`
- `PATCH /api/admin/products/:id`
- `POST /api/admin/products/:id/publish`
- `POST /api/admin/products/:id/archive`

10. Inventory Module

Responsibilities

- warehouse management
- inventory item current state
- inventory movement history
- low stock detection
- reservation and release logic
- manual stock adjustment

Critical rules

- inventory should primarily be tracked at variant level

- all stock changes must create `inventory_movements`
- manual adjustments require admin audit logging
- available stock = on_hand - reserved
- low-stock thresholds should be settings-driven

Key flows

- reserve stock during checkout or payment window
- release stock on payment failure/expiry
- deduct stock on payment success or dispatch depending on policy
- restock on return completion when applicable

11. Cart and Checkout Module

Responsibilities

- guest and user carts
- cart item pricing snapshots for UX
- coupon application preview
- checkout validation
- order draft creation
- payment initialization handoff

Critical rules

- never trust frontend totals
- backend recalculates totals during checkout
- backend determines eligibility for guest checkout
- backend returns action eligibility flags for cancel/return/review later

Checkout service responsibilities

- validate items and stock
- validate coupons/promotions
- calculate totals
- create order + order_items + order_addresses
- initialize payment
- reserve stock if policy requires
- create timeline history

12. Orders Module

Responsibilities

- order creation
- order state transitions
- fulfillment state transitions
- order notes
- cancellation requests
- return workflows
- customer and admin order views

Core order rules

- `order_status`, `payment_status`, and `fulfillment_status` are separate
- state transitions should be explicit and validated
- every important transition writes `order_status_history`
- snapshot product and address data into order tables

Admin controls

Admins may:

- confirm
- move to processing
- add notes
- cancel where allowed
- create or update shipment
- initiate refund workflows

All sensitive actions should be logged.

13. Payments Module

Responsibilities

- provider abstraction
- payment initialization
- webhook verification
- payment transaction history
- refund synchronization
- payment failure handling

Provider abstraction

Implement a `PaymentProviderAdapter` interface with operations like:

- initializePayment
- verifyPayment
- handleWebhook
- refundPayment

Critical rules

- webhook verification is authoritative
- do not trust frontend payment success
- store raw provider payloads carefully
- all payment state changes should write `payment_transactions`

Failure handling

- repeated payment failures may trigger alerts
- invalid webhook signatures should create `security_events`
- failed payment spikes should be visible in monitoring

14. Shipping and Fulfillment Module

Responsibilities

- shipping method management
- shipment creation
- shipment tracking updates
- warehouse linkage
- fulfillment queue support

Important rules

- allow multiple shipments per order for future split-shipment support
- shipment events should build timeline visibility
- customer-facing tracking pages consume backend shipment timelines

15. Promotions and Coupons Module

Responsibilities

- coupon management
- promotion rule evaluation
- eligibility calculation
- coupon redemption tracking

Rules

- coupon rules must be validated at checkout time
- usage limits must be concurrency-safe
- promotions should be centrally evaluated rather than scattered through controllers

16. Reviews Module

Responsibilities

- review creation
- review eligibility checks
- moderation state handling
- review listing and summaries

Rules

- review eligibility should be backend-driven
- verified purchase checks should use order/order_item state
- moderation actions should be admin-audited where necessary

17. Support Module

Responsibilities

- ticket creation
- support messaging
- internal note handling
- attachment handling
- SLA/reporting support later
- link support tickets to orders when relevant

Rules

- internal notes must not leak into customer-visible messages
- status changes should write `support_status_history`
- ticket creation and messaging should create timeline/activity entries

18. Notifications Module

Responsibilities

- email/SMS/push dispatch orchestration
- order confirmation emails
- shipment updates
- support notifications
- password reset / verification emails
- provider delivery event tracking

Pattern

- create logical `notifications` record
- enqueue delivery job
- record provider callbacks in `notification_events`

19. Content Module

Responsibilities

- banners
- CMS pages
- campaign content
- merchandising blocks later

Rules

- publishing state should be explicit
- content changes by admin should be audited

20. Customers Module

Responsibilities

- customer list and detail aggregation for admin
- customer order/support/review summaries

- customer account state actions
- customer activity timeline aggregation

Special note

This module is more of an aggregation/orchestration module than a pure table-owner module because customer detail pages need data from:

- users
- orders
- support
- reviews
- security_events
- risk_signals

21. Audit Module

Responsibilities

- general audit trail writes
- audit querying for admin explorer
- entity timeline support
- admin action logs

Rules

- append-only
- no hard edits of log records
- important admin actions should capture before/after where relevant

22. Security Module

Responsibilities

- login event analysis hooks
- suspicious event creation
- security event querying
- risk signal aggregation
- session governance hooks

Events that may create security records

- repeated login failures
- invalid admin access attempts
- suspicious refund patterns
- webhook signature failures
- coupon abuse patterns
- suspicious checkout behavior

23. Alerts and Incidents Module

Responsibilities

- alert generation
- alert assignment and resolution
- incident creation and linkage
- investigation support

Alert sources

- low stock
- payment failure spikes
- queue failures
- suspicious logins
- webhook failures
- large refund anomalies
- system exceptions

Incident usage

Incidents represent structured investigations that may link to:

- users
- orders
- payments
- alerts
- support tickets
- security events

24. Integrations Module

Responsibilities

- provider clients
- webhook adapters
- third-party retry logic
- integration health checks

Candidates

- payment gateways
- email providers
- SMS providers later
- storage providers
- analytics providers later

25. Jobs and Workers

Worker categories

- payment verification worker
- webhook processing worker
- email notification worker
- support notification worker
- low stock scan worker
- alert generation worker
- report generation worker later

Job design rules

- idempotent where possible
- status tracked in `job_runs`
- retry policy per job type
- poison/failure visibility for repeated failures

26. Reporting and Analytics Module

Responsibilities

- sales aggregation queries
- product performance aggregation
- support analytics

- refund/return analytics
- customer summaries
- dashboard KPIs

Rules

- do not overload live transactional endpoints with heavy reporting queries
- consider materialized views or scheduled summary tables later if needed
- reports should be permission-aware on admin side

27. API Design and Response Conventions

Conventions

- consistent route naming
- DTO-based validation
- explicit pagination envelopes
- explicit filter and sort contracts
- machine-readable error codes

Example response envelope

```
{
  "success": true,
  "data": {},
  "meta": {}
}
```

Error style

```
{
  "success": false,
  "error": {
    "code": "ORDER_NOT_ELIGIBLE_FOR_CANCELLATION",
    "message": "This order can no longer be cancelled."
  }
}
```

28. Validation Strategy

Use Zod or equivalent for:

- request body validation

- query validation
- param validation
- provider payload validation where practical

Validation should happen:

- before controller logic
- inside services where cross-table business rules are involved

29. Transactions and Consistency

Use Prisma transactions for operations such as:

- checkout order creation
- inventory reservation/release
- refund approval state change bundles
- support state + history updates
- admin role assignment + audit writes where necessary

Be careful not to place slow provider network calls inside long-running DB transactions.

30. Caching Strategy

Use Redis or similar for:

- session storage if applicable
- short-lived catalog caches
- rate limiting
- idempotency helpers
- queue backing

Avoid caching highly sensitive or rapidly mutating operational detail without invalidation strategy.

31. Webhooks and Idempotency

Webhooks

All inbound webhooks should:

- verify signatures
- write `webhook_events`
- be processed idempotently
- produce clear failure/retry behavior

Idempotency

Recommended for:

- payment verification
- refund sync
- webhook processing
- order-placement edge cases if payment providers retrigger callbacks

32. Observability and Monitoring

Logging

Use structured JSON logs with:

- request id
- user/admin id where available
- module
- action
- status
- timing
- error metadata

Metrics

Track:

- API latency
- error rates
- login failures
- payment failures
- low stock count
- queue backlog
- webhook failures
- support backlog
- alert counts

Error monitoring

Use Sentry or equivalent for:

- unhandled exceptions
- provider failures
- queue worker crashes
- high-severity operational errors

33. Health and Reliability Endpoints

Recommended endpoints:

- `GET /health`
- `GET /ready`
- `GET /metrics`
- optional `/admin/system/health` aggregation endpoint

34. Testing Strategy

Minimum layers

- unit tests for business rules
- integration tests for services + DB behavior
- webhook tests
- auth and permission tests
- checkout/order/payment flow tests
- refund/return/support flow tests

Highest-value test targets

- stock reservation consistency
- payment confirmation idempotency
- refund approval flows
- support visibility separation
- admin permission enforcement
- eligibility flags returned to frontend

35. Security Hardening

Minimum requirements:

- strong password hashing

- rate limiting
- secure cookie/session strategy or hardened JWT approach
- input validation
- provider webhook signature validation
- secure file upload validation
- role-based access control
- audit logs for sensitive admin actions
- secrets in environment variables only
- no raw provider secrets in logs

36. Multi-Store Readiness Strategy

Do not implement full tenancy now.

Prepare for later by:

- keeping modules store-ready
- using flexible settings scope
- keeping warehouses and inventory location-aware
- avoiding assumptions that only one operational unit can ever exist

Future `store_id` candidates:

- warehouses
- inventory_items
- shipping_methods
- promotions
- settings
- banners
- selected analytics scopes

37. Implementation Phases

Phase 1: Core commerce

- auth
- users
- roles/permissions
- catalog
- categories/brands
- inventory

- carts
- checkout
- orders
- payments
- shipping

Phase 2: Customer service and content

- reviews
- support
- notifications
- content
- customers aggregation

Phase 3: Governance and operational control

- audit
- security
- alerts-incidents
- system-settings
- jobs-workers
- health-observability

Phase 4: Reporting and optimization

- reports
- richer analytics
- cache optimization
- materialized reporting patterns later
- advanced fraud/risk heuristics later

38. AI Coding Agent Guidance

An AI coding agent should:

- implement module by module
- derive Prisma models from the approved schema pack
- keep service logic centralized
- use DTOs and validators consistently
- avoid inventing missing statuses or transitions
- wire audit/security/alert hooks into sensitive paths

- return frontend-friendly eligibility flags for customer actions
- keep admin APIs permission-gated and action-audited

39. Final Position

This backend implementation pack is designed as the execution bridge between:

- the database architecture
- the 72-screen admin frontend
- the 50-screen customer web frontend
- the 48-screen mobile frontend

It gives the platform a backend foundation that is:

- implementation-ready
- modular
- auditable
- operationally observable
- safe for sensitive commerce flows
- scalable enough for future multi-store evolution